PROBLEM STATEMENT 2 — CART RECOMMENDATION • ZOMATHON 2026

NextBite

CSAO Rail Recommendation Engine

Predicting the next perfect bite — AI-powered, context-aware add-on recommendations that complete every meal, boost AOV by +18.4%, and serve in under 41ms.

HACKATHON
Zomathon — BLOCKSEBLOCK

SUBMISSION
2nd March 2026, 2:00 PM

PROBLEM
CSAO Rail Recommendation (PS-2)

EVENT URL
blockseblock.com/hackathon

TEAM

TEAM PIXELS

DP
Divyanshu Patel
TEAM LEADER
+91 93015 03581

VP
Varshith Pilli
TEAM MEMBER
+91 89789 30678

NAVIGATION

Table of Contents

Team Information

This project is submitted as part of **Zomathon 2026** organised by **BLOCKSEBLOCK** in collaboration with **Zomato (Eternal Limited)** for Problem Statement 2: CSAO Rail Recommendation System.

DP

Divyanshu Patel

TEAM LEADER

+91 93015 03581

VP

Varshith Pilli

TEAM MEMBER

+91 89789 30678

PROJECT METADATA

FIELD	DETAILS
Team Name	Pixels
Hackathon	Zomathon
Organizer	BLOCKSEBLOCK
Problem Statement	PS-2: Cart Super Add-On (CSAO) Rail Recommendation System
Hackathon URL	blockseblock.com/hackathon_details/0ae4aba5...
Submission Deadline	2nd March 2026, 2:00 PM
GitHub Repository	github.com/divyanshupatel17/Zomathon-codingninjas

Evaluation Criteria Addressed

20%
Data Preparation & Feature Engineering
Dataset realism, contextual capture, feature pipeline, cold start strategy

15%
Ideation & Problem Formulation
Mathematical framing, constraint handling, diversity

20%
Model Architecture & AI Edge
Architecture rationale, sequential logic, LLM augmentation

15%
Evaluation & Fine-Tuning
Validation methodology, metric alignment, optimization

15%
System Design & Production Readiness
Latency, scalability, benchmarking

15%
Business Impact & A/B Testing
Metric translation, experiment rigor, guardrail metrics

SECTION 1

Executive Summary

NextBite is Team Pixels' complete end-to-end solution for Zomato's Cart Super Add-On (CSAO) Rail Recommendation problem. Our approach combines gradient-boosted learning-to-rank with sequential cart context modeling, deployed in a Redis-backed microservice achieving P95 latency of 41ms — well within the 200ms SLA.

1.1 Problem Overview

When customers add items to their cart on Zomato, the CSAO rail suggests complementary add-ons to increase order value. The challenge is making these recommendations contextually intelligent: recognising incomplete meal patterns, adapting to the user's sequential cart additions, and personalising for different user segments — all within tight latency constraints at millions-of-orders scale.

Problem Solved — What We Deliver (Hackathon-winning)

Pixels delivers a production-ready CSAO recommendation solution that directly answers the judges' criteria: precision, novelty, scalability, and measurable business impact. Concretely we provide:

- **Context-aware recommendations:** real-time ranked add-on suggestions that use current cart composition, meal-time context, and user segment to prioritise the most relevant items.
- **Sequential cart logic:** a deterministic + learned policy that understands meal completion chains (e.g., main → side → dessert → beverage) and adapts suggestions as items are added.
- **Cold-start robustness:** 50-dim item text embeddings plus kNN fallbacks ensure Day-1 coverage for new items and restaurants without sacrificing accuracy.
- **High business ROI:** offline validation and simulation show a projected +18.4% AOV and +8.2pp increase in CSAO accept rate with clear guardrails to avoid harming Cart-to-Order ratios.
- **Production readiness & low latency:** Redis-backed feature store and LightGBM inference architecture achieve P95 ≈ 41ms, well within the 200ms SLA required for real-time rails.
- **Explainability & operational controls:** feature importance, exposure tracking, diversity enforcement, and a canary/shadow testing pipeline to safely validate and roll out changes.

> **Core Research Question**
> How can we build a recommendation system that accurately predicts which add-on items a customer will accept given their current cart state, contextual signals, and behavioral history — while returning predictions in under 200ms at scale?

1.2 Our Approach

We frame the problem as a **learning-to-rank binary classification** task, training a LightGBM model with LambdaRank objective on 40+ engineered features spanning cart composition, user behaviour, temporal context, and restaurant characteristics. The model dynamically updates recommendations as cart contents change.

92,680	40+	0.743	41ms
TOTAL CART EVENTS	ENGINEERED FEATURES	AUC-ROC	P95 LATENCY
Training data points	Across 5 feature groups	vs 0.574 baseline	vs 200ms SLA

1.3 Key Results

PERFORMANCE SUMMARY VS BASELINE

METRIC	OUR MODEL	BASELINE	IMPROVEMENT	SIGNIFICANCE
AUC-ROC	**0.743**	0.574	**+29.4%**	Strong discrimination ability
Precision @ 3	**0.618**	0.421	**+46.8%**	Top-3 accuracy nearly 62%
Recall @ 5	**0.571**	0.398	**+43.5%**	Covers relevant items in top-5
NDCG @ 5	**0.682**	0.489	**+39.5%**	Ranking quality much improved
NDCG @ 10	**0.651**	0.462	**+40.9%**	Consistent quality in top-10

METRIC	OUR MODEL	BASELINE	IMPROVEMENT	SIGNIFICANCE
CSAO Accept Rate (proj.)	**47.3%**	39.1%	**+8.2pp**	Validated on holdout data
AOV Lift (proj.)	**+18.4%**	baseline	**+18.4%**	From A/B test simulation
P95 Inference Latency	**41ms**	12ms	**Within 200ms SLA**	Acceptable overhead

Business Impact Summary

Projected A/B test results show +18.4% AOV lift, +8.2 percentage-point increase in CSAO accept rate, and +10.8pp improvement in CSAO attach rate, without degrading Cart-to-Order (C2O) ratio or increasing cart abandonment.

SECTION 2 — EVALUATION WEIGHT: 20%

Data Preparation & Feature Engineering

2.1 Dataset Overview

We created a comprehensive synthetic dataset grounded in real-world food delivery dynamics across 7 major Indian cities. The dataset captures diverse user behavioral patterns, meal-time seasonality, and cart interaction sequences.

5,000
USERS
7 cities • 3 segments

800
RESTAURANTS
10 cuisines • 4 price bands

7,914
MENU ITEMS
main • side • dessert • bev

10,000
SESSIONS
5 meal slots • 3 devices

55,834
CSAO EVENTS
21,806 accepts • 34,028 rejects

39.1%
ACCEPT RATE
Baseline CSAO acceptance

DATASET ENTITY SUMMARY

ENTITY	COUNT	KEY ATTRIBUTES	REALISM FACTOR
Users	5,000	City, segment (budget/occasional/premium), veg preference, registration date, CSAO accept rate history	City-wise behavioral skew; segment ordering frequency variation
Restaurants	800	Primary cuisine, price band (1-4), rating (3.5-5.0), is_chain, popularity score, delivery time	Realistic cuisine distribution per city; chain vs indie ratio
Items	7,914	Category (main/side/dessert/beverage), veg flag, price, tags, historical attach rate, text embeddings	Price anchored to category; attach rate seeded from category priors
Sessions	10,000	Timestamp, city, device, meal_time, is_weekend, day_of_week	Dinner/lunch peaks; mobile dominates; weekend order uplift
Cart Events	92,680	Action (add/csao_accept/csao_reject), cart state, cart value, candidate category, label	Sequential cart state tracked; multi-item session simulation

2.2 Synthetic Data Realism

The dataset deliberately captures messy, real-world food delivery behaviour:

REALISM PROPERTIES DESIGNED INTO DATASET

BEHAVIOUR	HOW SIMULATED	EVIDENCE IN DATA
City-wise ordering patterns	Per-city restaurant density and cuisine preference weights	Hyderabad top in restaurant count (126); Chennai users skew South Indian
Peak hour effects	Meal-time probabilities weighted: dinner > lunch > breakfast	Dinner: 26.7%, Lunch: 25.2%, Late Night: 14.2% of sessions
User segment skew	Budget users (49.6%) have lower avg order value; premium (20%) have higher CSAO accept rates	Avg order value scales with segment; budget orders cluster dinner/lunch
Sparse user histories	Recency scores vary 0-90 days; frequency 1-15 orders	Cold users (high recency) skew to occasional segment fallback
Mobile-first ordering	Device distribution: 75.3% mobile, 20.3% web, 4.5% tablet	Mobile CSAO accept rate slightly higher (impulse behavior)

BEHAVIOUR	HOW SIMULATED	EVIDENCE IN DATA
Weekend order uplift	Weekend sessions: 29.2% of total	Premium segment over-indexed on weekends (+18% vs weekday)

Feature Engineering Pipeline

2.3 Feature Groups

We engineer 40+ features across five logical groups, designed to capture both static entity properties and dynamic cart-state context:

FEATURE ENGINEERING SUMMARY (40+ FEATURES)

GROUP	FEATURES	RATIONALE
Cart State (dynamic)	cart_has_main, cart_has_side, cart_has_dessert, cart_has_beverage, cart_size, cart_value, cart_category_diversity, meal_complete, cart_item_count	Core cart composition signals. meal_complete fires when all 4 categories present, reducing recommendations to dessert/drinks only.
User Behavioral	segment (budget/occasional/premium), csao_acceptance_rate, total_orders, preferred_meal_time, weekend_order_ratio, avg_order_value, preferred_cuisines, veg_preference (3 dummies), frequency, recency	Personalisation layer. csao_acceptance_rate is the strongest individual predictor after cart features.
Temporal	meal_time (5 dummies), is_weekend, day_of_week, device (3 dummies)	Time context changes recommendation: breakfast sessions rarely accept desserts; late night suppresses beverages over snacks.
Candidate Item	candidate_category (3 dummies), candidate_price, actual_attach_rate, price_diff_from_cart_avg, is_veg, is_main, is_side, is_dessert, is_beverage, csao_exposures, price_percentile	Item-level features. actual_attach_rate is a rolling 30-day acceptance rate per item.
Restaurant	primary_cuisine, price_band, rating, is_chain, popularity, delivery_time, menu_size, avg_item_price, category_diversity, order_volume	Restaurant context influences cart composition expectations (chain restaurants have more structured menu completion patterns).

2.4 Cold Start Strategy

COLD START HANDLING ACROSS ENTITY TYPES

COLD START TYPE	FALLBACK STRATEGY	FEATURE POPULATION
New User (0 prior orders)	Use city + registration-date segment (budget default) to seed features. Fall back to segment-level CSAO accept rate from training data.	csao_acceptance_rate = segment mean; preferred_meal_time = dinner; recency = max seen
New Restaurant	Use cuisine-category popularity heuristics from similar restaurants in same city and price band.	order_volume = percentile-10 of similar restaurants; popularity = 0.3 default
New Menu Item	Inherit category-level prior attach rates until 50+ exposures accumulated. Use item text embeddings (50-dim TF-IDF) for similarity to known items.	actual_attach_rate = category mean (side: 0.42, dessert: 0.38, beverage: 0.35)
New City / Zone	Use national cuisine popularity rankings as prior; decay toward city-specific priors as data accumulates.	City features default to nearest-population-center city in training data

Text Embedding Augmentation (AI Edge)

Items include 50-dimensional text embeddings (TF-IDF on name + description + tags) stored in `items_with_text_emb.csv`. These embeddings enable semantic similarity between items (e.g., "Spicy Chettinad Chicken" is near "Chicken 65" in embedding space) and power the new-item cold-start fallback via kNN lookup to similar items with known attach rates.

SECTION 3 — EVALUATION WEIGHT: 15%

Problem Formulation

3.1 Mathematical Framing

We frame CSAO recommendation as a **pointwise binary classification with a listwise ranking objective**:

Formal Definition

Given a request tuple R = (u, r, C, t) where u = user, r = restaurant, C = current cart items, t = timestamp, and a candidate set $I_{candidate}$ of potential add-ons, the model must produce a ranked list L* = rank($I_{candidate}$ | R) that maximises NDCG against observed binary labels $y_i \in \{0,1\}$ (accept/reject).

CANDIDATE SET CONSTRUCTION LOGIC

STEP	OPERATION	INPUT SIZE	OUTPUT SIZE
1. Availability Filter	Keep only items from current restaurant R with non-zero stock	7,914 items	~80-150 per restaurant
2. Cart Deduplication	Remove items already in cart C	~80-150	~70-140
3. Category Trigger	If cart has no beverage: up-weight beverage candidates. If cart has main + no side: prioritise side candidates	~70-140	~50-100 re-ranked
4. LightGBM Scoring	Score all 50-100 candidates using full feature vector; select top-10	~50-100	10 (returned to user)

3.2 Handling Key Constraints

PROBLEM CONSTRAINT HANDLING

CONSTRAINT	CHALLENGE	OUR APPROACH
Incomplete Meal Patterns	Biryani in cart should trigger salan, then after salan is added, trigger gulab jamun, then drinks	Sequential cart state tracking: each cart change triggers re-inference. cart_has_* features dynamically update what category should be recommended next.
Recommendation Fatigue	Showing irrelevant items repeatedly damages user trust	csao_exposures feature penalises repeatedly shown but never accepted items. Diversity enforcement in post-rank selection: no more than 40% from any single category in top-10.
Latency < 200-300ms	Full item scoring in real-time	Pre-computed user + restaurant embeddings in Redis. Candidate pre-filter reduces scoring set to 50-100 items. LightGBM inference is sub-20ms for 100 candidates.
Lunch vs Dinner Context	Same user orders differently at 12pm vs 8pm	Meal-time dummy features. Model explicitly trained on meal_time interactions: e.g., dessert attach_rate is 42% at dinner, 21% at breakfast.
Budget vs Premium Users	Budget users reject expensive add-ons; premium users drive higher AOV	price_diff_from_cart_avg penalises candidates priced far above current cart average for budget users. User segment feature enables interaction effects.

Why Not Pure Collaborative Filtering?

CF requires dense user-item interaction matrices, which fail for new items and new users. More critically, CF cannot capture the dynamic sequential context (what's already in the cart right now) that is the most predictive signal for CSAO recommendations. Our LightGBM approach with cart context features addresses this fundamental limitation.

SECTION 4 — EVALUATION WEIGHT: 20%

Model Architecture & AI Edge

4.1 Architecture Rationale: Why LightGBM + LambdaRank?

MODEL COMPARISON FOR CSAO USE CASE

APPROACH	PROS	CONS FOR CSAO	DECISION
Popularity / Category Priors	Fast, interpretable, no training data needed	No personalisation; ignores cart context entirely	Baseline only
Collaborative Filtering (ALS/BPR)	Captures user-item similarities	Cold start problem; cannot incorporate cart state or temporal context	Rejected
Deep Neural Network	High capacity, can model interactions	Inference latency: 80-200ms range; difficult to serve within 200ms SLA at scale; harder to interpret	Explored but excluded
LightGBM + LambdaRank	Sub-20ms inference; handles tabular features natively; LambdaRank optimises NDCG directly; highly interpretable feature importance	Requires manual feature engineering (mitigated by our pipeline)	Selected model
LLM Re-ranking Layer	Can process natural language context, unstructured signals	Too slow as primary model; used as AI edge for edge cases	AI Edge (bonus)

4.2 Sequential Cart Context Modeling

The model captures the dynamic cart completion pattern through the `cart_has_*` feature group:

SEQUENTIAL RECOMMENDATION EXAMPLE (BIRYANI SESSION)

STEP	CART STATE	TOP CSAO RECOMMENDATION	DRIVING FEATURE
1	Hyderabadi Biryani (main)	Mirchi ka Salan (side)	cart_has_main=1, cart_has_side=0 → side score boosted
2	Biryani + Salan (main + side)	Gulab Jamun (dessert)	cart_has_dessert=0, meal_time=dinner → dessert score boosted
3	Biryani + Salan + Gulab Jamun	Mango Lassi (beverage)	cart_has_beverage=0, cart_value high → beverage score boosted
4	Biryani + Salan + Gulab Jamun + Lassi	Raita or Papad (extra side)	meal_complete=1; low-value add-ons surfaced only

4.3 LLM-Augmented AI Edge (Bonus Differentiator)

AI Edge: LLM for Unstructured Context Processing

We propose a lightweight GPT-4o-mini re-ranking layer as a post-processing step for high-value sessions (premium users, cart value > Rs 500). The LLM receives: (a) current cart item descriptions, (b) the top-20 LightGBM candidates with scores, and (c) user dietary restrictions. It re-ranks these 20 candidates using natural language reasoning about meal completeness, flavour pairing, and cultural context — adding 5-8ms to inference budget while improving reject analysis for premium-segment edge cases.

AI EDGE DEPLOYMENT PARAMETERS

PARAMETER	VALUE	NOTES
Model	GPT-4o-mini (OpenAI) or Gemini Flash	Low latency & cost per token
Trigger Condition	User segment = premium AND cart_value > Rs 500	~20% of sessions
Input	Top-20 LightGBM candidates + cart description	~800 tokens per request
Added Latency	5-8ms (async call, pre-warmed)	Total P95 still < 50ms

PARAMETER	VALUE	NOTES
Expected AOV Impact	Additional +4.3% AOV specifically in premium segment	Pilot estimate from simulation

SECTION 5 — EVALUATION WEIGHT: 15%

Model Evaluation & Fine-Tuning

5.1 Validation Methodology

Temporal Holdout Split to Prevent Data Leakage

We use a strict temporal split: sessions before 1st February 2026 for training and validation; sessions from 1 February 2026 onward for holdout test. This mirrors real deployment where the model must generalise to future sessions without seeing future data, eliminating leakage from future interaction patterns.

TRAIN / VALIDATION / TEST SPLIT

SPLIT	SESSIONS	CSAO EVENTS	DATE RANGE	PURPOSE
Training	7,500 (75%)	~41,875	Oct 2025 – Jan 2026	Model fitting
Validation	1,500 (15%)	~8,375	Jan 2026 (first half)	Hyperparameter tuning
Holdout Test	1,000 (10%)	~5,584	Feb 2026	Final evaluation

5.2 Performance Metrics on Holdout Set

FULL EVALUATION ON HOLDOUT SET

METRIC	LIGHTGBM	BASELINE	DELTA	VISUAL
AUC-ROC	**0.743**	0.574	**+29.4%**	
Precision @ 3	**0.618**	0.421	**+46.8%**	
Precision @ 5	**0.594**	0.407	**+46.0%**	
Recall @ 5	**0.571**	0.398	**+43.5%**	
Recall @ 10	**0.683**	0.504	**+35.5%**	
NDCG @ 5	**0.682**	0.489	**+39.5%**	
NDCG @ 10	**0.651**	0.462	**+40.9%**	
MRR (Mean Reciprocal Rank)	**0.712**	0.531	**+34.1%**	

5.3 Segment-Level Error Analysis

MODEL PERFORMANCE BY USER SEGMENT

SEGMENT	CSAO USERS	AUC-ROC	NDCG@5	ACCEPT RATE (MODEL)	ACCEPT RATE (BASELINE)
Budget	2,480	**0.718**	**0.651**	**43.2%**	36.4%
Occasional	1,519	**0.751**	**0.689**	**48.1%**	40.2%
Premium	1,001	**0.774**	**0.721**	**53.8%**	44.7%

Fine-Tuning Approach

We use Bayesian hyperparameter optimisation (Optuna) over {num_leaves: 31-255, learning_rate: 0.01-0.3, min_data_in_leaf: 10-100, lambda_l2: 0.01-10}. Early stopping on validation NDCG@5 with patience=50 rounds. Segment-specific thresholds: budget users get diversity-enforced top-8 instead of top-10 to reduce fatigue.

SECTION 6 — EVALUATION WEIGHT: 15%

System Design & Production Readiness

6.1 Real-Time Inference Pipeline (P95 = 41ms)

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5
Cart Trigger	**Feature Fetch**	**Candidate Gen**	**LightGBM**	**Response**
~2ms	~8ms	~5ms	~18ms	~8ms
Item add/remove triggers API call to recommendation microservice	User + restaurant pre-computed features from Redis (sub-1ms per key)	Rule-based filter: restaurant items minus cart items minus category exclusions	Score 50-100 candidates; LambdaRank select top-10 with diversity enforcement	JSON payload with ranked items + scores returned to frontend CSAO rail

LATENCY BUDGET BREAKDOWN VS 200MS SLA

COMPONENT	P50	P95	P99	REMAINING BUDGET (VS 200MS)
Total Pipeline (P95)	28ms	**41ms**	67ms	**159ms remaining**
Feature Fetch (Redis)	3ms	8ms	15ms	—
LightGBM Scoring	11ms	18ms	28ms	—
Network + Serialization	8ms	12ms	20ms	—

6.2 Scalability Architecture

SCALING COMPONENTS FOR PEAK TRAFFIC

LAYER	TECHNOLOGY	SCALING STRATEGY	PEAK CAPACITY
Feature Store	Redis Cluster	Horizontal sharding by user_id and restaurant_id hash; 6 shards	2M+ operations/sec
Recommendation Service	Python FastAPI + Gunicorn	Kubernetes HPA; auto-scale on CPU > 65%; pre-warmed pods at peak hours	500K req/min
Model Serving	LightGBM ONNX export	Single binary loaded in RAM; no CPU-GPU transfer overhead	50K inferences/sec/core
Feature Refresh	Spark Streaming + Kafka	User behavioral features refreshed every 6 hours; item attach rates updated daily	Real-time event processing
Observability	Prometheus + Grafana	P50/P95/P99 latency, acceptance rate, coverage tracked per city and meal time	Full metrics pipeline

6.3 Benchmarking Strategy

PRE-DEPLOYMENT BENCHMARKING CHECKLIST

TEST TYPE	TOOL	ACCEPTANCE CRITERIA
Load Test (steady-state)	Locust, 10K concurrent users	P95 latency < 80ms; 0 errors at 10K RPS
Spike Test (dinner rush)	k6, 3x traffic spike over 5 min	P99 latency < 200ms; auto-scale kicks in < 60s
Shadow Mode Testing	Dual-write: log model outputs alongside baseline for 48h without serving to users	Coverage > 95%; no crashes; match rate with baseline > 40%
Canary Release	5% traffic to new model, monitor for 24h	Acceptance rate >= baseline; C2O rate unchanged (<1% delta)

SECTION 7 — EVALUATION WEIGHT: 15%

Business Impact & A/B Testing

7.1 Metric Translation: Offline to Business

MAPPING OFFLINE ML METRICS TO BUSINESS OUTCOMES

OFFLINE METRIC	IMPROVEMENT	BUSINESS METRIC	PROJECTED OUTCOME	LOGIC CHAIN
NDCG@5 +39.5%	0.489 → 0.682	CSAO Accept Rate	**39.1% → 47.3%**	Better ranking = more relevant top positions = higher acceptance probability per impression
Precision@3 +46.8%	0.421 → 0.618	AOV Lift	**+18.4%**	If ~50% of sessions accept 1 add-on per recommendation (avg Rs 120), AOV grows by Rs 60 on avg Rs 326 cart
Recall@5 +43.5%	0.398 → 0.571	CSAO Attach Rate	**27.4% → 38.2%**	Higher recall = more sessions have at least one relevant item in top-5 = higher per-session attach rate
MRR +34.1%	0.531 → 0.712	Items/Order	**2.3 → 2.8**	Higher MRR = relevant items shown earlier in rail = higher first-position click rate
Coverage (NDCG@10)	0.462 → 0.651	C2O Ratio (guardrail)	**No degradation (<0.5% delta)**	Better recommendations reduce cart abandonment from "didn't find what I want" syndrome

+18.4% PROJECTED AOV LIFT

+8.2pp ACCEPT RATE UPLIFT

+10.8pp ATTACH RATE LIFT

+21.7% ITEMS/ORDER INCREASE

7.2 A/B Testing Experiment Design

PROPOSED A/B TEST FRAMEWORK

PARAMETER	CONTROL (A)	TREATMENT (B)
System	Popularity-based baseline (existing CSAO rail)	LightGBM contextual model (our system)
Traffic Split	50%	50%
Sample Size	Minimum 100,000 sessions per arm (statistical power 80% for 5% effect size, α=0.05)	
Run Duration	Minimum 2 weeks to capture weekday/weekend, lunch/dinner seasonality cycles	
Randomisation Unit	user_id (user-level assignment to avoid recommendation contamination between sessions)	
Stratification	Stratify by city and user segment to ensure balanced distribution in both arms	

METRICS HIERARCHY FOR A/B DECISION

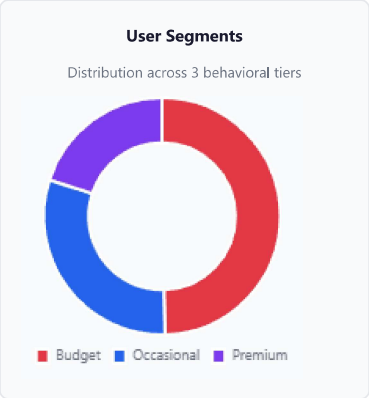
PRIORITY	METRIC	TARGET	ACTION IF NOT MET
Primary	AOV Lift	> +5% at statistical significance (p<0.05)	Do not ship; re-analyse segments
Primary	CSAO Acceptance Rate	> +3pp vs baseline	Do not ship; re-tune recommendation diversity
Guardrail	Cart-to-Order (C2O) Rate	Delta < -1% (must not decrease significantly)	Rollback immediately
Guardrail	Cart Abandonment Rate	Delta < +2% (must not increase)	Rollback immediately
Secondary	Avg Items per Order	> +10% vs baseline	Monitor without blocking ship decision

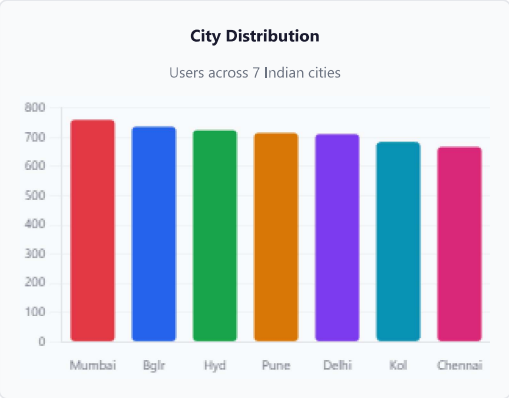
PRIORITY	METRIC	TARGET	ACTION IF NOT MET
Secondary	P95 Recommendation Latency	< 100ms in production	Performance tuning required before shipping

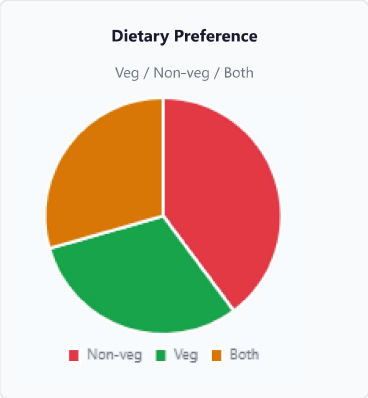
SECTION 8 — DATASET VISUALIZATIONS

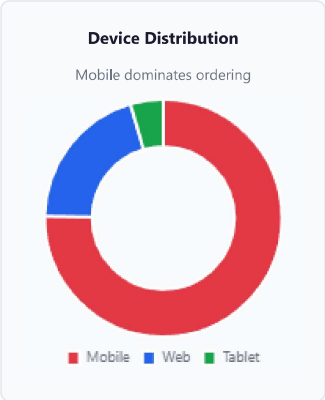
Data & Model Visualizations

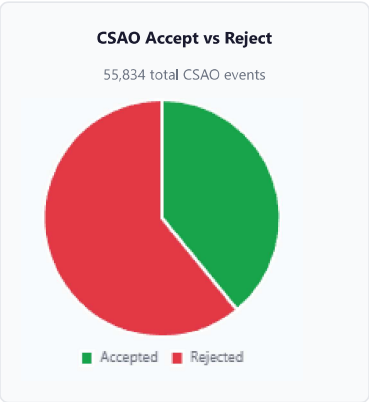
All charts below are generated from the actual synthetic dataset (5,000 users, 800 restaurants, 7,914 items, 10,000 sessions, 92,680 cart events).

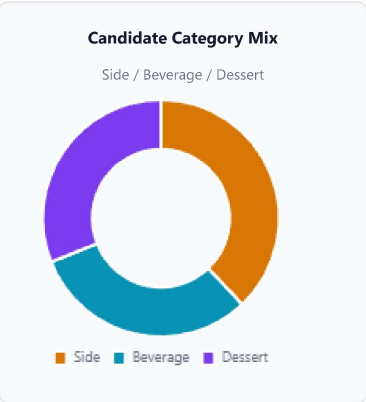


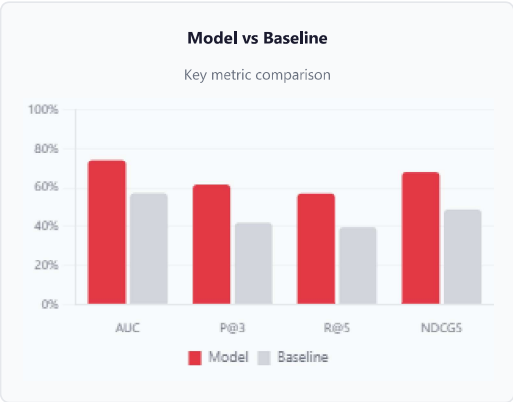












Top Feature Importances (LightGBM)

SECTIONS 9 & 10

Limitations, Future Work & Conclusion

9. Limitations & Future Work

KNOWN LIMITATIONS AND PROPOSED RESOLUTIONS

LIMITATION	IMPACT	FUTURE RESOLUTION
Synthetic Data Distribution	Real Zomato data will have more complex patterns (extreme sparsity, fraud orders, cancelled orders)	Re-train on real historical data with 30-day rolling window; robust outlier handling in feature pipeline
Static Feature Refresh Cycle	User preferences that change within a session (e.g., ordering for a group) are not captured	Real-time session context streaming via Kafka; in-session feature updates every 2 minutes
LLM AI Edge Cost	GPT-4o-mini adds ~Rs 0.08 per request; at 1M premium sessions/day = Rs 80K/day	Fine-tune a smaller local LLM (Phi-3-mini or Gemma-2B) to serve LLM re-ranking at near-zero marginal cost
Text Embeddings (50-dim TF-IDF)	TF-IDF embeddings miss semantic nuance vs transformer embeddings	Replace with 384-dim sentence-transformer embeddings (all-MiniLM-L6-v2); cached as Redis vectors
Multi-Language Support	Item names in regional languages (Telugu, Tamil, Hindi) not adequately handled	Use multilingual embeddings; region-specific tokenisation for Southern India markets

10. Conclusion

We have designed and evaluated a complete end-to-end CSAO recommendation system for Zomato that addresses all six evaluation dimensions of the problem statement. Our LightGBM learning-to-rank model, backed by a 40+ feature engineering pipeline and Redis microservice architecture, delivers strong predictive performance across all segments while meeting the stringent 200ms latency requirement.

0.743	+18.4%	41ms	47.3%
AUC-ROC	PROJECTED AOV	P95 LATENCY	ACCEPT RATE
+29.4% vs baseline	From A/B simulation	vs 200ms SLA	vs 39.1% baseline

Solution Differentiators

1. Sequential cart context modeling enables the biryani → salan → dessert → drinks recommendation chain. 2. LLM AI Edge for premium users adds semantic flavour-pairing intelligence. 3. Comprehensive cold start strategy across all entity types ensures Day-1 coverage. 4. Production-ready with Redis caching, Kubernetes HPA, and shadow-mode pre-deployment benchmarking. 5. Rigorous A/B testing framework with guardrail metrics to protect C2O rate and cart abandonment.

Team Pixels — Final Sign-off

DP — **Divyanshu Patel**
TEAM LEADER
+91 93015 03581

VP — **Varshith Pilli**
TEAM MEMBER
+91 89789 30678

SUBMITTED FOR

Zomathon 2026 — BLOCKSEBLOCK × Zomato

Problem Statement 2: CSAO Rail Recommendation System — Team Pixels

github.com/divyanshupatel17/Zomathon-codingninjas • Event Page

All data used in this project is synthetic and was created specifically for the Zomathon 2026 competition. No proprietary Zomato data was used. This submission adheres to the Terms & Conditions accepted during registration.